

GPU Bundle Adjustment: Damping Menu, Block Preconditioning, and Basin Stability

Cumulative results

2026-08-31

Executive summary

Three lines of work are reported. (i) The multi-shift *damping menu* was tested against its own absence for the first time: it does **not** produce better minima (median +0.02% over 23 BAL problems) and at equal quality it is about **12% slower** on a typical problem. Its value is *insurance* — a 3.5–8× payoff on the subset where a single shift selects a bad basin, and the elimination of run-to-run bimodality. (ii) The *block-congruence preconditioner* was generalised to the shared-intrinsics reduced space, removing the constraint that kept it out of the mapper; it now delivers ~6.5% end-to-end wall at neutral quality. (iii) The long-standing ladybug-1723 failure was diagnosed as *basin sensitivity*, *not a defect*, closing it as an investigation.

1 The damping menu: with versus without

Each outer iteration prices a menu of damping values $\sigma_l = \lambda \cdot 10^{l-\text{grid_down}}$, $l = 0..L-1$ from a *single* Krylov sweep via the ζ -recurrence, then scores the candidate steps by true nonlinear cost. $L=5$ is the shipped width; $L=1$ removes the menu entirely (classical Levenberg–Marquardt). All arms use the block preconditioner and the v6 policy.

dataset	with menu ($L=5$)			without ($L=1$)			quality	iso-q.
	final	wall	mv	final	wall	mv	Δ	speed
dubrovnik-135	4.8206e+05	3.5s	410	4.8214e+05	3.0s	364	+0.02%	0.88x
dubrovnik-173	3.8146e+05	3.7s	315	3.8151e+05	3.2s	277	+0.01%	0.87x
dubrovnik-88	3.6715e+05	2.2s	228	3.6173e+05	2.0s	257	-1.48%	0.68x
final-1936	5.0499e+06	29.5s	168	5.0499e+06	25.7s	146	+0.00%	1.01x
final-3068	1.9696e+06	8.1s	95	2.2288e+06	20.8s	410	+13.16%	8.07x
final-4585	1.3578e+07	167.5s	472	1.8270e+07	108.1s	324	+34.55%	4.94x
final-93	1.5836e+05	0.7s	151	1.5836e+05	0.6s	96	-0.00%	0.87x
insta360-3086	1.1112e+06	32.2s	665	1.1428e+06	3.3s	68	+2.85%	5.54x
ladybug-1197	3.6995e+05	4.3s	1111	3.7200e+05	5.4s	1272	+0.56%	4.10x
ladybug-1469	4.3267e+05	4.4s	852	4.3526e+05	6.4s	1290	+0.60%	3.52x
ladybug-1723	7.6342e+05	10.8s	644	5.0168e+05	7.5s	488	-34.28%	0.24x
ladybug-49	1.3669e+04	0.2s	246	1.3668e+04	0.2s	234	-0.01%	0.82x
ladybug-598	1.8088e+05	1.9s	675	1.7989e+05	1.8s	712	-0.55%	0.50x
ladybug-810	2.2410e+05	3.0s	1138	2.2390e+05	2.4s	890	-0.09%	0.78x
muellcontainer-90	2.9691e+05	1.8s	442	2.9692e+05	1.6s	403	+0.00%	1.10x
trafalgar-126	1.0511e+05	0.8s	229	1.0510e+05	0.7s	235	-0.01%	0.86x
trafalgar-201	1.1291e+05	1.0s	237	1.1293e+05	0.9s	255	+0.02%	0.89x
trafalgar-257	1.1816e+05	1.2s	328	1.1832e+05	1.0s	315	+0.14%	0.98x
venice-1672	2.2310e+06	29.2s	295	2.2189e+06	26.4s	334	-0.54%	0.75x
venice-1778	2.0338e+06	32.2s	491	2.0350e+06	26.3s	363	+0.06%	0.88x
venice-52	2.4181e+05	2.9s	855	2.4201e+05	2.0s	353	+0.09%	1.02x
venice-52-noisy	1.2439e+06	2.5s	556	1.2442e+06	1.7s	108	+0.02%	1.12x
venice-89	3.0412e+05	7.8s	2826	3.0563e+05	6.5s	2616	+0.50%	2.46x
total		351s			257s			

dataset	with menu ($L=5$)			without ($L=1$)			quality	iso-q.
	final	wall	mv	final	wall	mv	Δ	speed

Table 1: All 23 BAL problems. **mv** = matvecs. **quality** Δ is the without-menu final relative to with-menu (positive = menu better). **iso-q. speed** compares time to a common target (the harder of the two finals, which both arms reach); > 1 means the menu is faster.

What the aggregate says

quality outcome	n	median iso-quality speed
tied ($ \Delta \leq 0.05\%$)	9	$0.88\times$
menu gives better quality	9	$3.52\times$
menu gives worse quality	5	$0.68\times$
all	23	$0.89\times$

Endpoint quality is a wash: median $+0.02\%$, menu better on 9, worse on 5, tied on 9 of 23. Removing the menu is -26.7% total wall. At *equal quality* the menu is median $0.89\times$ — i.e. roughly 11% **slower**. It is not routinely faster.

Why it is still worth keeping: the payoff is asymmetric

dataset	with menu	without	menu is
final-3068	$8.07\times$		
insta360-3086	$5.54\times$		
final-4585	$4.94\times$		
ladybug-1197	$4.10\times$		
ladybug-1469	$3.52\times$		
venice-89	$2.46\times$		

The menu costs roughly 12% on problems where it changes nothing, and repays $3.5\text{--}8\times$ where a single shift commits to a poor step. The tail is what matters: without the menu **final-4585** lands in a basin 34.6% worse, and under the block preconditioner **ladybug-1723** spreads by 698% .

2 Against the baselines on BAL

The shipped configuration ($L=5$, block preconditioner) against the GPU baseline (Caspar) and CPU Ceres on the same 23 problems and the same objective. Both crossing directions are given: where our final is worse, the baseline’s time to *our* final is the honest comparison, not endpoint walls.

dataset	ours ($L=5$, block)		Caspar		quality	crossing	
	final	wall	final	wall	Δ	ours $\rightarrow$ C	C $\rightarrow$ ours
dubrovnik-135	4.8206e+05	3.5s	4.7478e+05	3.1s	$+1.53\%$	never	0.2s
dubrovnik-173	3.8146e+05	3.7s	3.7896e+05	3.5s	$+0.66\%$	never	1.3s
dubrovnik-88	3.6715e+05	2.2s	3.5755e+05	2.3s	$+2.69\%$	never	0.1s
final-1936	5.0499e+06	29.5s	5.0530e+06	27.3s	-0.06%	8.5s ($3.22\times$)	never
final-3068	1.9696e+06	8.1s	2.6317e+06	1.0s	-25.16%	0.8s ($1.22\times$)	never
final-4585	1.3578e+07	167.5s	1.1035e+07	15.4s	$+23.04\%$	never	2.2s
final-93	1.5836e+05	0.7s	1.5844e+05	1.9s	-0.05%	0.3s ($5.89\times$)	never
insta360-3086	1.1112e+06	32.2s	1.0828e+06	6.0s	$+2.62\%$	never	0.2s
ladybug-1197	3.6995e+05	4.3s	4.5533e+05	0.3s	-18.75%	0.5s ($0.63\times$)	never
ladybug-1469	4.3267e+05	4.4s	6.7672e+05	0.2s	-36.06%	0.4s ($0.54\times$)	never
ladybug-1723	7.6342e+05	10.8s	6.5345e+05	0.2s	$+16.83\%$	never	0.1s

dataset	ours ($L=5$, block)		Caspar		quality Δ	crossing	
	final	wall	final	wall		ours $\rightarrow$ C	C $\rightarrow$ ours
ladybug-49	1.3669e+04	0.2s	1.3618e+04	0.5s	+0.37%	never	0.3s
ladybug-598	1.8088e+05	1.9s	1.8186e+05	0.2s	-0.54%	0.7s (0.31 $\times$)	never
ladybug-810	2.2410e+05	3.0s	2.2434e+05	0.6s	-0.10%	1.4s (0.41 $\times$)	never
muellcontainer-90	2.9691e+05	1.8s	2.9754e+05	2.0s	-0.21%	0.4s (5.45 $\times$)	never
trafalgar-126	1.0511e+05	0.8s	1.0666e+05	1.1s	-1.45%	0.2s (4.74 $\times$)	never
trafalgar-201	1.1291e+05	1.0s	1.1400e+05	1.5s	-0.96%	0.4s (3.72 $\times$)	never
trafalgar-257	1.1816e+05	1.2s	1.2041e+05	1.6s	-1.87%	0.4s (4.71 $\times$)	never
venice-1672	2.2310e+06	29.2s	2.4636e+06	20.0s	-9.44%	4.6s (4.32 $\times$)	never
venice-1778	2.0338e+06	32.2s	2.1116e+06	29.0s	-3.68%	2.6s (11.15 $\times$)	never
venice-52	2.4181e+05	2.9s	2.7964e+05	1.7s	-13.53%	0.3s (6.46 $\times$)	never
venice-52-noisy	1.2439e+06	2.5s	1.2840e+06	2.1s	-3.12%	0.3s (8.13 $\times$)	never
venice-89	3.0412e+05	7.8s	3.1050e+05	3.2s	-2.05%	0.3s (10.20 $\times$)	never

Table 2: **ours $\rightarrow$ C** is our time to reach Caspar’s final (speed-up in brackets); **C $\rightarrow$ ours** is Caspar’s time to reach ours, and reads *never* where Caspar cannot match us.

We end better on 16 of 23 (median -0.54%) and Caspar never reaches our final on 16 of 23; where we win, we reach Caspar’s answer $4.52\times$ faster in median. **The seven losses are material and concentrated:** on **final-4585** Caspar reaches 1.10×10^7 in 15.4s against our 1.36×10^7 after 167.5s — 23% worse at eleven times the wall, and Caspar reaches our final in 2.2s. **ladybug-1723** (+16.8%), **dubrovnik-88** and **insta360-3086** follow the same pattern. Several ladybug sets show the opposite asymmetry: a better final but a crossing below $1\times$ — a better answer, reached more slowly.

Ceres as a quality reference

Ceres is not a wall competitor here — it grinds 1000 iterations where the GPU solvers take seconds — but it answers a question neither GPU solver can: *how good is the minimum actually available*. Two columns are therefore kept strictly separate. The quality column uses only runs that reached a genuine stopping point (their own tolerance, or the iteration ceiling); truncated runs are excluded from it, because a capped run’s last cost is not a minimum. The crossing columns remain valid for capped runs, since crossings occur at iteration 2–5, long before any cap.

dataset	iters	status	Ceres min	vs ours	vs Caspar	C $\rightarrow$ ours	C $\rightarrow$ Caspar
dubrovnik-135	1000	max-iter	4.5600e+05	-5.41%	-3.95%	6.5s (0.53 $\times$)	8.2s (0.38 $\times$)
dubrovnik-173	1000	max-iter	3.7641e+05	-1.32%	-0.67%	3.6s (1.04 $\times$)	3.6s (0.98 $\times$)
dubrovnik-88	1000	max-iter	3.4763e+05	-5.32%	-2.77%	3.0s (0.73 $\times$)	6.7s (0.34 $\times$)
final-1936	7	capped	5.0497e+06	–	–	555.1s (0.05 $\times$)	434.3s (0.06 $\times$)
final-3068	28	capped	1.8447e+06	–	–	156.3s (0.05 $\times$)	135.4s (0.01 $\times$)
final-4585	0	no solve		Ceres completed no iteration inside the cap			
final-93	12	converged	1.5836e+05	-0.00%	-0.05%	2.4s (0.29 $\times$)	1.7s (1.14 $\times$)
insta360-3086	58	capped	1.0827e+06	–	–	18.8s (1.71 $\times$)	154.7s (0.04 $\times$)
ladybug-1197	127	converged	3.6873e+05	-0.33%	-19.02%	42.9s (0.10 $\times$)	15.6s (0.02 $\times$)
ladybug-1469	387	capped	1.0452e+06	–	–	never	never
ladybug-1723	73	converged	4.5295e+05	-40.67%	-30.68%	13.0s (0.82 $\times$)	14.9s (0.02 $\times$)
ladybug-49	1000	max-iter	1.3534e+04	-0.98%	-0.62%	0.3s (0.92 $\times$)	0.3s (2.03 $\times$)
ladybug-598	133	converged	6.7702e+05	+274.29%	+272.27%	never	never
ladybug-810	107	converged	2.2343e+05	-0.30%	-0.40%	15.4s (0.20 $\times$)	14.6s (0.04 $\times$)
muellcontainer-90	19	converged	2.9691e+05	-0.00%	-0.21%	2.3s (0.78 $\times$)	1.2s (1.67 $\times$)
trafalgar-126	111	converged	1.0354e+05	-1.50%	-2.93%	0.6s (1.29 $\times$)	0.6s (1.76 $\times$)
trafalgar-201	57	converged	1.0813e+05	-4.23%	-5.15%	1.1s (0.96 $\times$)	1.1s (1.39 $\times$)
trafalgar-257	86	converged	1.2068e+05	+2.13%	+0.23%	never	never
venice-1672	40	converged	2.2240e+06	-0.31%	-9.73%	199.5s (0.15 $\times$)	131.3s (0.15 $\times$)
venice-1778	38	capped	1.9644e+06	–	–	55.5s (0.58 $\times$)	55.5s (0.52 $\times$)
venice-52	879	capped	2.4035e+05	–	–	15.3s (0.19 $\times$)	4.9s (0.36 $\times$)

dataset	iters	status	Ceres min	vs ours	vs Caspar	C→ours	C→Caspar
venice-52-noisy	582	converged	1.2419e+06	-0.16%	-3.28%	12.5s (0.20×)	4.5s (0.48×)
venice-89	479	capped	3.0291e+05	—	—	17.8s (0.44×)	2.7s (1.18×)

Table 3: **C→ours** is Ceres time to reach our final; the bracketed factor is our wall divided by it, so < 1 means the GPU solver got there first. Quality columns are blank for truncated runs by construction.

Across the 15 runs with a usable minimum, Ceres ends -0.33% against ours and -2.77% against Caspar — it finds a **materially better minimum**. The crossings show the other half: the GPU solvers reach any given quality first, by a median factor of 1.9, and the margin widens sharply with size (on **final-1936** and **final-3068** Ceres needs $0.05\times$, i.e. the GPU is roughly $20\times$ faster to the same cost). **Limitation.** On **final-4585** Ceres could not complete a single iteration within the 600s cap, so no CPU reference exists there. That is unfortunate precisely because **final-4585** is where our solver loses hardest to Caspar; the comparison that would settle whether that is a basin problem or a solver problem is the one measurement out of reach at this budget.

3 Stability: the decisive argument

The endpoint table conceals the strongest effect. Repeating the *same solve on the same input* three times:

problem	arm	three repeats	spread
final-4585	with menu ($L=5$)	1.3587e+07, 1.3593e+07, 1.3594e+07	0.1%
final-4585	without menu ($L=1$)	1.8270e+07, 1.8270e+07, 1.4932e+07	22.4%
ladybug-1723	without menu, block	—	697.7%
ladybug-1723	with menu, block	—	15.1%
ladybug-1723	without menu, diag	—	0.1%
ladybug-1723	with menu, diag	—	0.0%

On **final-4585** the no-menu arm returned two *bit-identical* results and one far apart — two discrete basins, not measurement scatter. Narrowing the menu degrades repeatability monotonically ($L=5 \rightarrow L=3 \rightarrow L=1$).

4 Menu width: is $L=5$ the right constant?

A single-run sweep suggested $L=3$ beat the shipped $L=5$ on the large cold problems. At $N=3$ that claim does not survive.

dataset	$L=5$ med.	spread	$L=3$ med.	spread	$L3$ vs $L5$	single-run said
final-3068	1.9478e+06	5.9%	1.7320e+06	1.1%	-11.08%	-12.45%
final-4585	1.3589e+07	0.1%	1.4178e+07	10.2%	+4.33%	-5.27%
insta360-3086	1.0859e+06	2.7%	1.0862e+06	1.8%	+0.03%	-2.43%
ladybug-1197	3.6991e+05	1.5%	1.4712e+06	18.4%	+297.70%	+309.72%

Verdict: keep $L=5$. Only one of four confirms; **final-4585** flips sign (-5.27% single-run becomes $+4.33\%$ at $N=3$) and **ladybug-1197** is catastrophic. The spread columns corroborate the stability thesis independently: $L=3$ is markedly less repeatable than $L=5$ on three of four datasets.

5 Block-congruence preconditioner

Replaces the diagonal equilibration $DSD + \sigma I$ with a per-camera congruence: build the exact Schur diagonal block $S_c = H_{cc,c} - \sum_{\text{obs}} WV^{-1}W^\top$, factor $S_c = LL^\top$, and solve $L^{-1}SL^{-\top} + \sigma I$. Crucially the shift still enters as σI , so the shared Krylov space — and therefore the whole menu — survives. A general preconditioner M would turn the shift into σM^{-1} and destroy it.

Shared intrinsics: the constraint that blocked adoption

The mapper shares intrinsics across calibration groups, so the CG works in the reduced space $[6 n_{\text{cam}} \mid 3 n_{\text{calib}}] = B^\top S B$ and the 9×9 blocks cannot be used directly. They are split into a 6×6 pose block per camera plus a 3×3 calibration block per group. Cross-camera terms within a group and pose–calibration cross terms are dropped (both off-block).

muell GBA replays (shared intrinsics)	
dumps run	22/22
Cholesky fallbacks	0
better final cost	18/22
quality (median)	-0.001%
wall (median)	-6.5%
total wall	83.6s $\rightarrow$ 75.3s (-10.0%)

End-to-end mapper run (493 images)

Per-call replay neutrality has previously failed to predict end-to-end behaviour, so adoption rests on a full sequence.

run	arm	wall	regs	points	sum_sq	mean px
run 1	off	1005s	491	315988	4.524413e+06	1.1100
run 1	blockred	922s	491	315712	4.521385e+06	1.1102
	delta	-8.3%	+0	-276	-0.067%	+0.0002
run 2	off	954s	491	315644	4.517137e+06	1.1097
run 2	blockred	910s	491	315791	4.520278e+06	1.1102
	delta	-4.6%	+0	+147	+0.070%	+0.0005

Magnitude versus direction. The baseline itself varies 5.3% run to run (1005s vs 954s), which is the same order as the effect — so a single pair could not bound the size, and the first run’s -8.3% overstated it. The mean is -6.5% . The *direction* is nonetheless solid: every block run (910s, 922s) is faster than every baseline run (954s, 1005s), so the two distributions do not overlap.

Against the baselines: full mapper sequence

A separate controlled set (same database and images, two repeats per backend, all run on one day) places the two GPU backends against CPU Ceres end-to-end. These are *not* the same runs as the table above and should not be differenced against it.

backend	repeats (min)	mean	vs Caspar
Caspar	18.2, 17.9	18.1 min	1.00×
Mfree	18.8, 18.7	18.8 min	1.04×
Ceres	119.9, 120.6	120.3 min	6.7×

Both GPU backends complete the 493-image sequence in about 18–19 minutes against Ceres’ 120; the GPU advantage over CPU Ceres (6.7×) dwarfs every effect measured in this report. Caspar and the matrix-free solver are within a few percent of each other. **Caution on the sum_sq column.** sum_sq is a sum over observations, so it tracks point count. Run 1 shows -0.067% with 276 *fewer* points; run 2 shows $+0.070\%$ with 147 *more*. The sign follows the point count, not the solver. Mean reprojection error, which normalises, is flat in both. **Quality is neutral; the wall saving is the result.**

Largest problem: monocular Fuchsberg gba.234

8,788 cameras and 16.75M observations, converted from a fisheye rig so that a single-camera baseline can run it. This was the one problem where the solver previously *lost* to the baseline.

arm	wall	final	vs baseline	time to baseline final
baseline (Caspar)	85.4s	3.00357e+07	—	—
diagonal	180.8s	3.00264e+07	-0.03%	118.2s (0.72×)
block	94.3s	3.00208e+07	-0.05%	41.0s (2.08×)

The block preconditioner turns a $0.72\times$ loss into a $2.08\times$ win: matvecs fall from 2601 to 570 and the Krylov phase from 100.6s to 22.1s, against roughly 10s of extra factor-build cost. The fear that the factor build would dominate at scale was wrong — it grows with cameras while the Krylov saving grows with observations.

6 ladybug-1723: closed as basin sensitivity

This dataset had blocked adoption of the preconditioner as an unexplained regression. The decisive experiment perturbs only the *starting point* (cameras and points) by $\leq 10^{-15}$ relative — about four ULP — leaving the observations, and hence the minimum, untouched.

path	original	seed 1	seed 2	seed 3	spread
diag	4.5758e+05	4.5758e+05	4.5758e+05	4.5758e+05	0.0%
block	4.6196e+05	4.5981e+05	5.5927e+05	4.7756e+05	21.6%

The diagonal path returns a **bit-identical** result from four different starting points; the block path scatters by 21.6%, matching its run-to-run spread. The problem is *chaotic* under block scaling and contractive under diagonal scaling. There is no defect to fix, which explains why a conditioning guard and a per-camera hybrid both failed — both treated a stability property as a conditioning error. It is now a documented variance case.

7 The damping floor is inert on warm problems

On `gba_234`, 55 of 60 accepted steps sat exactly on the damping floor $\lambda_{\min} = 10^{-7}$, suggesting the policy wanted damping the menu could not express. Lowering the floor by six decades produces **bit-identical** cost trajectories.

floor	wall	final	matvecs	trajectory
10^{-8}	94.2s	3.00208e+07	570	reference
10^{-10}	94.3s	3.00208e+07	571	bit-identical
10^{-12}	94.4s	3.00208e+07	571	bit-identical
10^{-14}	94.4s	3.00208e+07	570	bit-identical

λ does descend to 10^{-13} when permitted, but below 10^{-7} the damping is numerically inert ($S + \lambda I \equiv S$). This yields a cheap **diagnostic**: if dropping the floor six decades changes nothing bitwise, the problem is in the undamped regime and all λ -policy work is dead weight on it.

8 Negative results and retractions

Recorded because each cost real time and each failed the same way — a delta attributed to a mechanism without first measuring the baseline’s own spread.

claim	outcome
“Narrowing the menu fixes ladybug-1723” (−39.6%)	Refuted. Measured against a single unusually bad baseline draw. At $N=3$, $L=3$ is <i>worse</i> (+7.5%) than $L=5$ (+3.1%).
“The block failure is kernel non-determinism (81 atomics/obs)”	Refuted. An atomics-free camera-major kernel still scatters 13.2%. The kernel is correct and modestly faster, but the cause was elsewhere.
“ $L=3$ beats the shipped $L=5$ on cold problems”	Refuted at $N=3$. final-4585 flips sign; only 1 of 4 confirms.
“Block-red improves quality end-to-end (−0.067%)”	Withdrawn. An artefact of 276 fewer points in a sum over observations; normalised error is flat.
“The menu is $2.04\times$ faster”	Corrected. That used the no-menu final as the target, which is a biased yardstick. Against a common target the menu is median $0.89\times$, i.e. $\sim 12\%$ <i>slower</i> .

9 The menu-degeneracy gate

A follow-on question — *does the shift grid adapt as λ shrinks?* — exposed that it recenters but never rescales: once λ falls below the spectrum of S , the five shifted systems collapse (candidates agreeing to 10^{-8} while still costing five residual passes each). The degeneracy is *per-checkpoint*: shallow checkpoints collapse while deep checkpoints in the same outer iteration discriminate strongly. The fix gates candidate scoring on the spread of the ζ -recurrence predictions $\text{preds}[l]$ — free, available before any scoring — and preserves canonical evaluation order unconditionally, so a live menu takes exactly the ungated path. Two earlier designs were refuted by measurement: a step-norm proxy (steps differ by 2.2% while costs agree to 10^{-8}) and score-extremes-first (+380% wall on one dataset from evaluation *reordering* alone, with the gate never firing).

OCA_MENU_GATE=1e-2 vs off, 23 BAL problems		
candidate evaluations (median)	−62%	
wall (median)	−10.7%	
final cost (median)	+0.0000%	17/23 tied, 8 bit-identical
wall regressions > 5%	none	(the criterion that rejected v2)
mapper (22 replays)	bit-identical	e2e neutral

The one alarming sweep number (ladybug-1197 +23.5%) did not reproduce at $N=3$ (+0.46% median); the gate does widen that dataset’s spread (0.05% $\rightarrow$ 0.97%). Tolerance 10^{-3} fails (+60% wall on the same dataset); 10^{-2} is the validated setting. Recommended for cold/benchmark solves; default off in code (bit-compatible).

10 Recommendations

setting	recommendation	basis
Menu width L	keep 5	$N=3$ refutes $L=3$; narrower is less repeatable.
OCA_BLOCKEQ + OCA_BLOCK_CM	adopt for the mapper	−6.5% end-to-end (mean of two pairs; non-overlapping distributions), identical registrations, 22/22 replays, 0 fallbacks.
OCA_LAM_FLOOR	leave at default (8)	Bit-identical across six decades; useful only as a diagnostic.
ladybug-1723	document, do not chase	Basin sensitivity; no defect exists to fix.

Where to spend effort next. Global BA is 31% of mapper wall (306s of 1005s over 50 calls), so warm-solve tuning is capped at roughly 5% end-to-end and is not worth a configuration split. Three independent results this cycle — the no-menu bimodality, the chaos of ladybug-1723 under block scaling, and the finding that the menu’s entire value lies in arbitrating between candidate steps rather than finding better ones — all point

at **basin selection** on cold problems as the mechanism worth attacking. That is likely worth more than any percentage-level gain measured here.